Module 9 — Redis

> Redis is the default answer for caching, distributed locks, session stores, and rate limiting. Interviewers want caching strategies, invalidation, and the correctness pitfalls of distributed locks.

9.1 Caching Fundamentals & Strategies

Why Interviewers Ask This

Caching is the highest-leverage performance tool and the source of the two hardest problems in CS ("cache invalidation and naming things"). They test whether you pick the right strategy and handle staleness/failures.

Core Concept

Redis is an in-memory key-value store (sub-ms latency). Cache = keep hot data close to compute to avoid slow backends.

Strategies

- **Cache-Aside (Lazy loading)** — app checks cache; miss → load from DB → populate cache. Most common. App owns cache logic. Risk: first-request miss latency, stale data until TTL/eviction.
- **Read-Through** — cache library loads from DB on miss transparently.
- **Write-Through** — write to cache **and** DB synchronously → cache always fresh, higher write latency.
- **Write-Behind (Write-Back)** — write to cache, async flush to DB → fast writes, risk of loss on crash.
- **Write-Around** — write to DB only; cache populated on later reads (good for write-heavy, rarely-read data).

ASCII — Cache-Aside

```
 read:  app -> cache? HIT -> return
                      MISS -> DB -> put in cache (TTL) -> return
 write: app -> DB -> INVALIDATE (delete) cache key
```

Real Production Example (Spring)

```
@Cacheable(value="products", key="#id")
Product get(Long id) { return repo.findById(id).orElseThrow(); }

@CacheEvict(value="products", key="#p.id")
Product update(Product p) { return repo.save(p); }
```

Trade-offs / Common Mistakes

- **Invalidation**: prefer **delete on write** over update-in-place (avoids race where a stale write wins). "Cache-aside + evict" is the safe default.
- **Stampede / thundering herd**: many misses hit the DB at once when a hot key expires → use locks/`@Cacheable` sync, request coalescing, or jittered TTL.
- **Cache penetration**: repeated misses for non-existent keys → cache negative results (short TTL) or bloom filter.
- **Cache avalanche**: many keys expire simultaneously → randomize/jitter TTLs.
- Caching without TTL; caching user-specific data under a shared key.

Interview Q / Follow-ups

- Cache-aside vs write-through vs write-behind — trade-offs.
- How do you invalidate cache safely on writes? *(delete, not update.)*
- Cache stampede/penetration/avalanche — what are they and mitigations?

- How to keep cache consistent with the DB? *(TTL + evict-on-write; accept eventual consistency; or write-through.)*

9.2 TTL & Eviction

- **TTL** (`EXPIRE`) bounds staleness and memory. Always set one for caches.
- **maxmemory-policy** eviction: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction`. Pick LRU/LFU for caches; `noeviction` for durable stores (writes fail when full).

Interview Q

Why set TTL; which eviction policy for a pure cache; LRU vs LFU.

9.3 Distributed Lock

Why Interviewers Ask This

Distributed locks are subtly incorrect if done naively — a favorite trap.

Core Concept

Coordinate mutually exclusive work across instances (e.g. run a scheduled job once). Basic: `SET key uuid NX PX 30000` (set if not exists + expiry). Release **only if you own it** via a Lua compare-and-delete (never a plain `DEL` — you might delete someone else's lock after your TTL expired).

```
if redis.call('get', KEYS[1]) == ARGV[1]
then return redis.call('del', KEYS[1]) else return 0 end
```

Pitfalls / Trade-offs

- **TTL too short** → lock expires mid-work → two holders. **Too long** → slow recovery on crash.
- Single-node Redis lock isn't safe under failover; **Redlock** (multi-node) is debated (Kleppmann's critique) — for strict correctness use a fencing token or a real consensus system (ZooKeeper/etcd). For "best-effort mutual exclusion," single-node + fencing is usually fine.
- Use **Redisson** (`RLock`) in Java — handles renewal (watchdog) and safe release.

ASCII

```
SET lock:job <uuid> NX PX 30000  -> acquired? do work
work must finish < TTL (or renew via watchdog)
release: Lua { if value==uuid then DEL }
```

Interview Q / Follow-ups

- How to implement a distributed lock correctly (NX+PX+owner check)?
- Why is `DEL` on release dangerous?
- What is Redlock and its criticism? What are fencing tokens?

9.4 Session Store

Externalize HTTP sessions to Redis (Spring Session) so any instance can serve any request → stateless app tier, no sticky sessions, survives restarts. Alternative to JWT for stateful web apps; enables central logout/revocation.

Interview Q

Session-in-Redis vs JWT; why externalize sessions (horizontal scaling).

9.5 Rate Limiter

Core Concept

Protect APIs from abuse/overload. Common algorithms: - **Fixed window** — count per window; simple but bursty at boundaries. - **Sliding window (log/counter)** — smoother, more accurate. - **Token bucket** — allow bursts up to bucket size, refill at rate (most common; Bucket4j/Redis). - **Leaky bucket** — constant outflow.

Redis implements these atomically (INCR+EXPIRE, sorted sets, or Lua) so limits are shared across instances.

ASCII — Token Bucket

```
tokens refill at R/sec up to capacity C
request: tokens>=1 ? consume 1, allow : reject (429)
```

Interview Q / Follow-ups

- Fixed vs sliding window vs token bucket — trade-offs.
- Why do it in Redis (shared, atomic) rather than per-instance?
- How to return 429 with `Retry-After`?

Module 9 — One-Page Cheat Sheet

Topic	Key point
Cache-aside	app-managed; miss→DB→populate; evict on write
Write-through	sync to cache+DB; fresh, slower writes
Write-behind	async to DB; fast, risk loss
Invalidation	delete key on write (not update)
Stampede	lock/coalesce + jittered TTL
Penetration	cache negatives / bloom filter
Avalanche	randomize TTLs
TTL/eviction	always TTL; allkeys-lru/lfu for caches
Distributed lock	SET NX PX + owner-check Lua release; Redisson watchdog
Session store	Spring Session → stateless app tier
Rate limit	token bucket in Redis (atomic, shared)

Module 9 — Top Interview Questions

1. Caching strategies and when to use each.
2. How do you keep cache consistent with the DB / invalidate safely?
3. Cache stampede/penetration/avalanche and mitigations.
4. Implement a correct distributed lock; why is naive release unsafe?
5. Redlock and its criticism; fencing tokens.
6. Session in Redis vs JWT.
7. Rate limiting algorithms; why Redis.
8. Eviction policies; LRU vs LFU.
9. Why always set a TTL?

10. `@Cacheable` / `@CacheEvict` semantics in Spring.

Module 9 — Common Mistakes

- No TTL → stale/unbounded memory.
- Updating cache in place instead of evicting → stale races.
- `DEL` release without owner check → deleting others' locks.
- Caching per-user data under a shared key.
- Relying on single-node Redis lock for strict correctness.

Module 9 — Mock Interview

1. *"DB is hammered when a hot cache key expires."* → cache stampede; lock/coalesce loads, `@Cacheable(sync=true)`, jitter TTL.
2. *"Run a nightly job exactly once across 5 instances."* → distributed lock (`SET NX PX`) or Redisson `RLock`; ensure TTL > job time or use watchdog.
3. *"Keep the product cache consistent after updates."* → evict key on write, short TTL, accept eventual consistency (or write-through).
4. *"Rate-limit an API to 100 req/min/user across instances."* → token bucket in Redis (atomic INCR/Lua), 429 + Retry-After.
5. *"Attackers query random non-existent IDs, bypassing cache."* → cache penetration; cache negative results with short TTL / bloom filter.

Next → Module 10: Database.